

Figure 13.4 TG circuit discussed in Ex. 13.2.

Series Connection of TGs

Consider the series connection of the CMOS transmission gates shown in Fig. 13.5. The equivalent digital model is also depicted in this figure. As seen in Figs. 10.17, 10.21, and the associated discussion, the capacitance on each MOSFET's source/drain (assuming triode operation) is $C_{ox}/2$. The total capacitance on each internal node in a series connection of TGs is then the sum of the oxide capacitances from each adjacent PG, that is, $C_{oxn} + C_{oxp}$. The delay through the series connection of TGs can be estimated using

$$t_{PHL} = t_{PLH} = 0.7 \cdot N \cdot (R_n || R_p)(C_{load}) + 0.35 \cdot (R_n || R_p)(C_{oxn} + C_{oxp})(N)^2 \quad (13.2)$$

The first term in this equation is simply the time needed to charge C_{load} through the sum of the TG effective resistances, while the second term in the equation describes the RC transmission line effects.

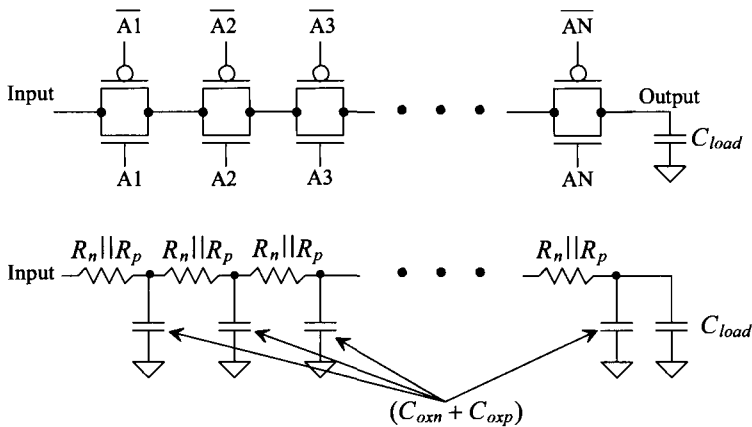


Figure 13.5 Series connection of TGs with digital model.

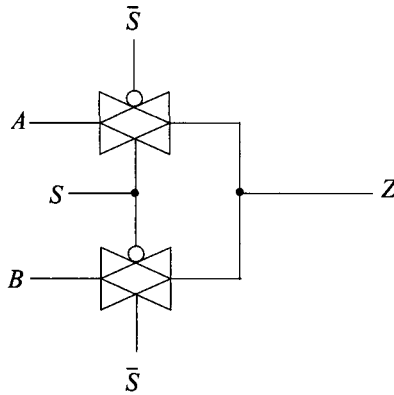


Figure 13.6 Path selector.

13.2 Applications of the Transmission Gate

In this section we present some of the applications of the TG.

Path Selector

The circuit shown in Fig. 13.6 is a two-input path selector. Logically, the output of the circuit can be written as

$$Z = AS + B\bar{S} \quad (13.3)$$

When the selector signal S is high, A is passed to the output while a low on S passes B to the output.

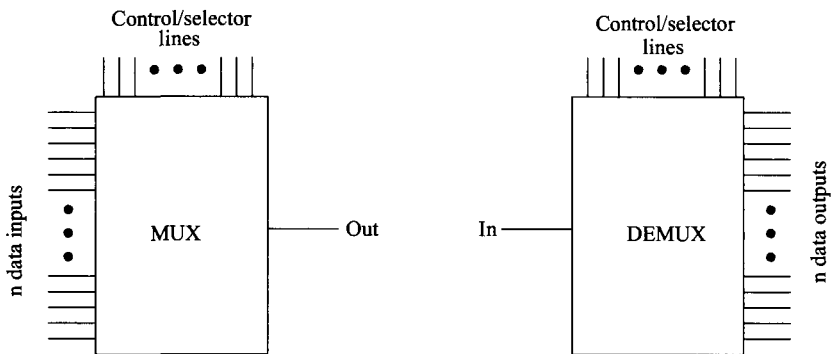


Figure 13.7 Block diagram of MUX/DEMUX.

This same idea can be used to implement multiplexers/demultiplexers (MUX/DEMUX). Consider the block diagrams of a MUX and DEMUX shown in Fig. 13.7. The number of control lines is related to the number of input lines by

$$2^m = n \quad (13.4)$$

where n is the number of inputs (outputs) to the MUX (DEMUX) and m is the number of control lines. A 4-to-1 MUX/DEMUX is shown in Fig. 13.8. Note that the MUX is bi-directional; that is, it can be used as a MUX or a DEMUX. The logic equation describing the operation of the MUX is given by

$$Z = A(S1 \cdot S2) + B(S1 \cdot \overline{S2}) + C(\overline{S1} \cdot S2) + D(\overline{S1} \cdot \overline{S2}) \quad (13.5)$$

Figure 13.9 shows the transistor-level implementation of the circuit in Fig. 13.8. Notice how the PMOS are grouped together (and laid out in the same n-well) while the NMOS are grouped together.

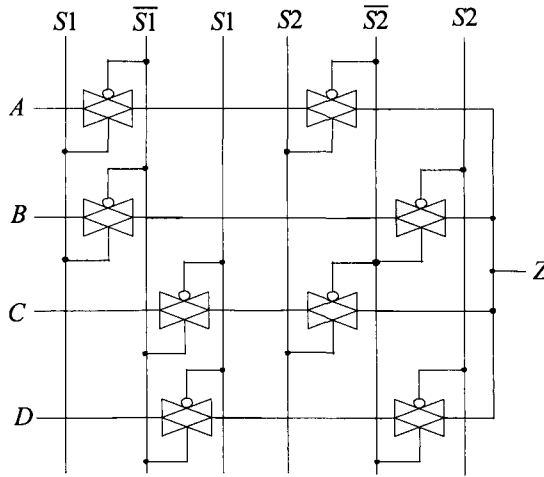


Figure 13.8 Circuit implementation of a 4-to-1 MUX.

Figure 13.10a shows an NMOS PG implementation of the 4-to-1 MUX. The pass transistor implementation is simpler, using fewer transistors, at the price of a threshold voltage drop from input to output when the input is a high (V_{DD}). A simplified version of the circuit of Fig. 13.10a is shown in Fig. 13.10b. Here the MOSFETs connected to $S2$ and $\overline{S2}$ are combined to reduce the total number of MOSFETs used. The reduction of the total number of MOSFETs used can be extended to an n -input (output) MUX (DEMUX) (see Fig. 16.43). Again, it should be remembered that a DEMUX can be formed using the circuits of Figs. 13.8 or 13.10 by switching the inputs with the outputs.

Static Gates

The TG can be used to form static logic gates. Consider the OR gate shown in Fig. 13.11. To understand the operation of the gate, consider the case when both A and B are low. Under these circumstances the pass transistor, M1 is off, and the TG is on. Since the input B is low, a low is passed to the output. If A is high, M1 is on and A is passed to the output. If B is high and A is low, B is passed to the output through the TG. If both A and B are high, the TG is off and M1 is on passing A , a high, to the output.

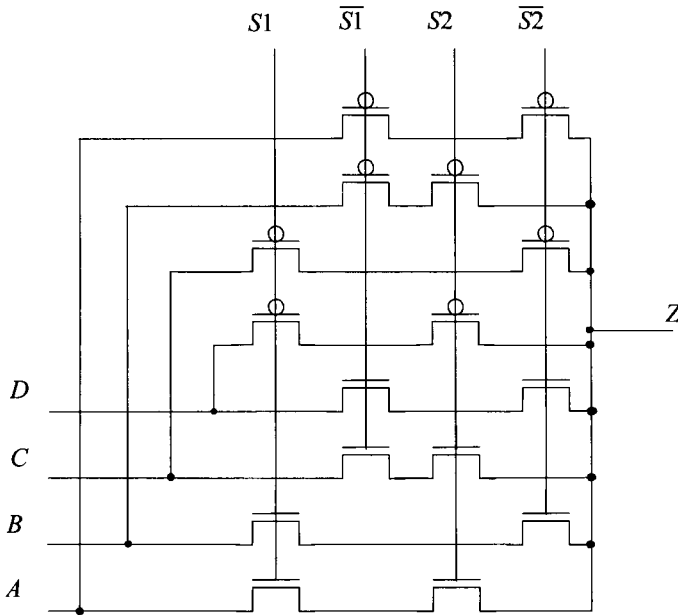


Figure 13.9 Transistor implementation of Fig. 13.8.

Figure 13.12 shows an XOR and an XNOR gate made using TGs. Consider the XOR gate with both A and B low. Under these circumstances, the top TG is on and its output is connected to A , a low. If both inputs are high, the bottom TG connects the output to $\bar{A}$, again a low. If A is high and B is low, the top TG is on and the output is connected to A , a high. Similarly, if A is low and B is high, the bottom TG is on and connects the output to $\bar{A}$, a high.

13.3 Latches and Flip-Flops

Basic Latches

Consider the set-reset latch (SR latch) shown in Fig. 13.13 made using NAND gates. The logic symbol and truth table are also shown in this figure. Consider the case when S is high and R is low. Forcing R low causes Q to go high. Since S is high and Q is high, the $\bar{Q}$ output is low. Now consider the case when both S and R are low. Under these circumstances, the latch's outputs are both high. This latch can easily be designed and laid out with the techniques of Ch. 12 (see also Fig. 15.6).

An alternative implementation of the SR latch is shown in Fig. 13.14 using NOR gates. Consider the case when S is high and R is low. For the NOR gate, a high input forces the output of the gate low. Therefore, the $\bar{Q}$ output is low whenever the S input is high. Similarly, whenever the R input is high, the Q output must be low. The case of both inputs being high causes both Q and $\bar{Q}$ to go low, or in other words the outputs of the latch are no longer complements.

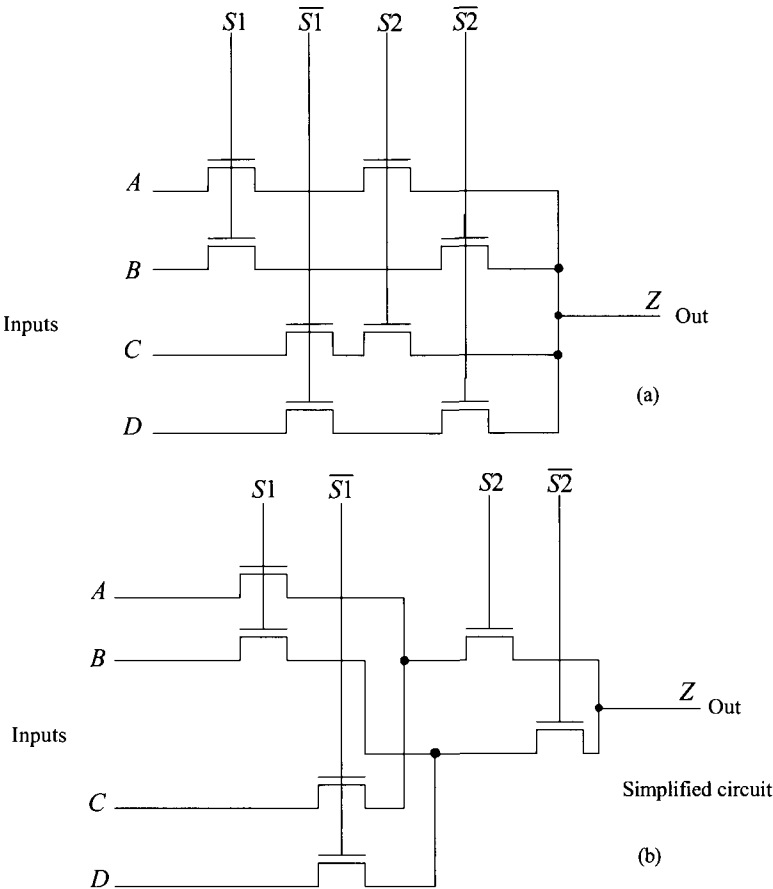


Figure 13.10 MUX/DEMUX using pass transistors.

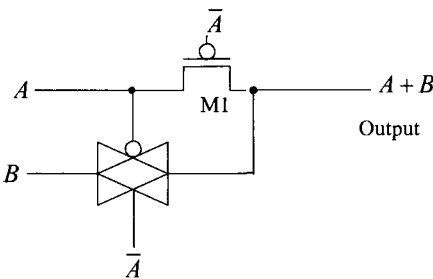


Figure 13.11 TG-based OR gate.

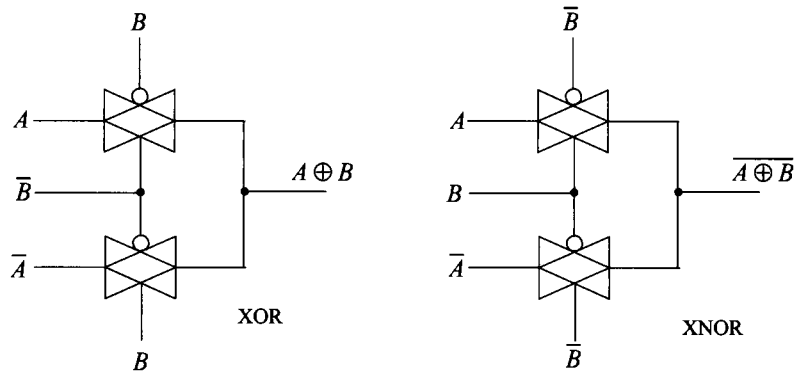


Figure 13.12 TG implementation of XOR/XNOR gate.

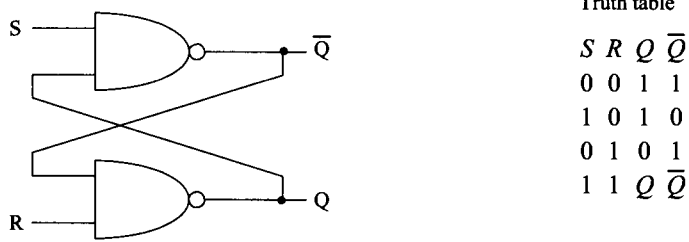


Figure 13.13 Set-reset latch made using NAND gates.

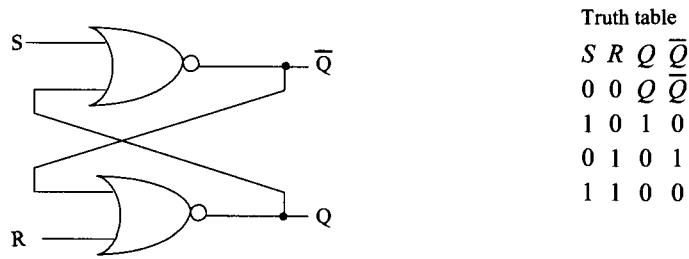


Figure 13.14 Set-reset latch made using NOR gates.

An Arbiter

One of the uses of the NAND latch is in an arbitration circuit (called an arbiter, Fig. 13.15). This circuit can be very useful in asynchronous circuit design or in clock synchronization circuits (see Ch. 19). To understand the operation of the arbiter in Fig. 13.15, let's consider the case when both In1 and In2 are low. Both NAND gates' outputs are high and the two inverters' outputs, that is Out1 and Out2, are low. When In1 goes high, the output of X1 goes low, while the output of X2 remains high. This causes Out1 to go high and Out2 to remain low. Notice that with the output of X1 going low the power supplied to the PMOS in the inverter connected to Out2 is removed (making it impossible for Out2 to go high). When In2 goes high, while In1 is already high, the fact that the output of X1 is a low keeps the output of X2 high and Out2 low. When In1 goes low, with In2 high, Out1 goes low and Out2 goes high. The usefulness of this circuit occurs when In1 and In2 transition high at nearly the same times. The way the inverters are powered by the NAND gates makes it impossible for both Out1 and Out2 to go high at the same time. *Thus an arbiter is useful for determining which input arrives first.*

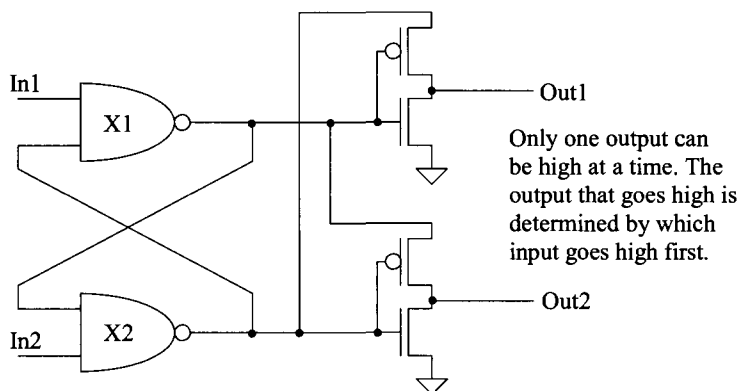


Figure 13.15 An arbiter made using NAND gates.

To ensure that the arbiter makes the decision quickly (there isn't a long delay or *metastability* when the two inputs are arriving at the same time), two inverters can be added in series with the NAND gate outputs. This addition of inverters simply increases the gain of the NAND gates (their VTCs get steeper at the switching point, see Fig. 12.3 in the last chapter).

Flip-Flops and Flow-through Latches

A flow-through latch is a storage circuit whose output changes with a clock signal level (so a flow-through latch is often called a level-sensitive latch). A flip-flop (FF) is a storage circuit that changes states on the rising or falling edge of a clock signal.

Most FFs in CMOS IC design are based on the cross-coupled inverters seen in Fig. 13.16. Also seen in this figure are the VTCs for the two inverters. It's characteristics are drawn with its input on the x-axis and its output on the y-axis (the normal curves as